

PILOS: Scalable Large-Subgraph Matching by Online Spectral Filtering

Konstantinos Skitsas
Aarhus University

Davide Mottin
Aarhus University

Panagiotis Karras
U. of Copenhagen & Aarhus U.

Abstract—*Subgraph matching* seeks all the occurrences of a *query graph* inside another graph. As it reduces to *subgraph isomorphism*, it is NP-hard. Current methods reduce the computation by *filtering* the candidates on which they run *subgraph isomorphism*. Nevertheless, when the query is large, the number of candidates grows rapidly, rendering current methods largely ineffective in pruning and incapable to answer even within one hour. A primary reason for this ineffectiveness is their inability to effectively consider the query graph structure in the computation.

In this paper, we propose PILOS, a novel matching algorithm that substantially improves the filtering phase of a typical matching algorithm and computes up to 60% fewer candidates for verification. PILOS uses (i) an *offline* light-weight index-based phase, which leverages the top graph Laplacian eigenvalues of query and data node neighborhoods to reduce candidates via *neighborhood filtering* and (ii) an *online* phase, which further prunes candidates stored in an auxiliary data structure; both phases apply the *interlacing theorem* on graph Laplacian spectra. Our thorough experimental study shows that, on average, PILOS resolves queries in 19% less time and leaves 23% fewer unresolved queries after a lapse of 10 minutes than the best previous work.

Index Terms—Subgraph Matching, Spectral Filtering

I. INTRODUCTION

Subgraph matching seeks all isomorphic occurrences of a *query subgraph* within a large graph; it finds application in chemistry [1], [2], bioinformatics [3]–[8], social network analysis [9]–[12], computer vision [13], [14], pattern recognition [15], [16], and elsewhere [17]–[21]. As the problem is NP-hard, several algorithms seek efficient heuristics [22].

Filter-and-verification (FV) techniques [23]–[31] build large *offline indexes* on the data graph that filter candidate query answers before verifying them for subgraph isomorphism. While offline indexing does not affect query time, there are limitations in the time and memory devoted for indexing [32]; more indexed information enables better pruning, yet the index size cannot scale to include all possible patterns [33]. On another extreme, *direct enumeration* algorithms explore the graph to *directly* retrieve results [3], [29], [34], [35]. The best direct enumeration algorithms outperform the best FV algorithms [36], highlighting the importance of the verification step. As filter-and-verification solutions are strong in filtering and direct enumeration solutions in verification, the two may be also combined [36]. Recently, state-of-the-art *preprocessing enumeration* algorithms [37]–[41] build a lightweight *auxiliary data structure* tuned to the query for pruning and verification tailored to query node labels, degrees, and neighborhoods; they add nodes passing the filtering step

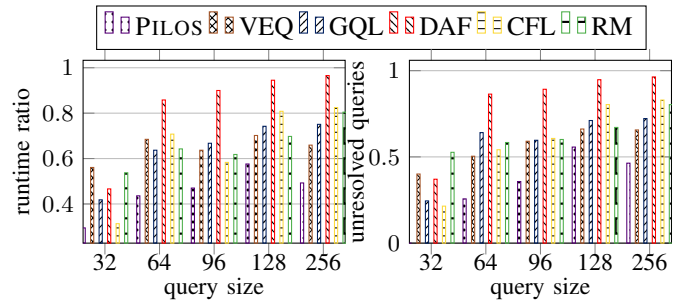


Figure 1: Query evaluation within 600s, all data sets combined; *left*: average runtime ratio over max runtime; *right*: average unresolved query ratio over max unresolved, up to 256 nodes.

to an interlinked *candidate space*. Yet, they typically employ only basic filtering rules, resulting in large candidate spaces. Combining preprocessing enumeration with an *offline* index increases pruning power [42]. Still, such methods base their filtering on simple structural features. While most small-graph queries (of up to 32 nodes) are thus solvable in less than a second [43], [44], the verification cost grows exponentially with query size, calling for more sophisticated filtering. To this end, *spectral* descriptors, which capture graph structures in a compact numerical form [45], [46], offer an attractive possibility to filter candidates and eschew costly verifications.

In this paper, we introduce PILOS,¹ a subgraph matching algorithm that first filters candidates by *lightweight* index-based pruning using graph Laplacian spectra and then applies *online spectral refinement* to further reduce candidates stored in an auxiliary data structure. Contrary to previous work that used spectral filtering only offline [47], PILOS applies it both offline and online, using the spectrum of a *fixed-size* subgraph around each node. In effect, it passes fewer candidates to verification, where it uses state-of-the-art methods [37], [38], [43], [48]. **Figure 1** shows our combined results on several datasets (Section V); PILOS stands out as the only method that resolves all queries of size 32 on all datasets; it resolves *more* queries, *faster* than the state of the art.

II. SUBGRAPH MATCHING

A *labelled graph* is a triple $G : (V, E, L)$ on a set of nodes V , a set of edges $E \subseteq V \times V$, and a labelling function $L : V \rightarrow \Sigma$ that assigns to each node a label from a label set Σ . The *neighborhood* $\mathcal{N}(u) = \{v \in V | (u, v) \in E \vee (v, u) \in E\}$ of a

¹From Greek φίλος, ‘felt’ (filter material).

node u is the set of nodes such that there exists an edge (u, v) or (v, u) . The *degree* $d(u)$ of a node $u \in V$ is the size of its neighborhood $d(u) = |\mathcal{N}(u)|$. The *label-neighborhood* of a node u is the multiset $\mathcal{N}_\Lambda(u) = \{L(v) | (u, v) \in E\}$ containing the labels of nodes adjacent to u . Likewise, the k -hop neighborhood of a node u , $\mathcal{N}_k(u)$, is the set of nodes reachable from u via at most k edges. An *induced subgraph* of a graph G over a set of nodes S is the graph $G[S] = (S, E_S, L)$ where $E_S = \{(u, v) \in E | u, v \in S\}$ is the set of edges in G that have both endpoints in S .

A graph $G_1 : (V_1, E_1, L_1)$ is **isomorphic** to $G_2 : (V_2, E_2, L_2)$, or $G_1 \equiv G_2$, if there exists a *bijective mapping function* $f : V_1 \rightarrow V_2$ such that for each node $v \in V_1$ there is a node $f(v) \in V_2$ with $L_1(v) = L_2(f(v))$, and for each edge $(u, v) \in E_1$ there is an edge $(f(u), f(v)) \in E_2$. A graph $G_1 : (V_1, E_1, L_1)$ is **subgraph isomorphic** to $G : (V, E, L)$, or $G_1 \sqsubseteq G$, if there exists a subgraph of G isomorphic to G_1 .

A *subgraph matching query*, expressed by a labelled graph $Q : (V_q, E_q, L_q)$ with set of nodes V_q , set of edges $E_q \subseteq V_q \times V_q$ and labelling function $L_q : V_q \rightarrow \Sigma$ on a graph G , seeks subgraphs of G isomorphic to Q , with answers $\mathcal{A}(Q) = \{G' \sqsubseteq G | Q \equiv G'\}$.

Problem 1 (Subgraph Matching). *Given a graph $G : (V, E, L)$ and a query $Q : (V_q, E_q, L_q)$ find all subsets $E' \subseteq E$ forming a subgraph $G' = (V', E', L')$ isomorphic to the query.*

We assume G to be *undirected*, i.e., $(u, v) \in E$ implies $(v, u) \in E$, however, our discussion readily adapts to the directed case, as well as to *edge-labelled* graphs.

The *adjacency matrix* $\mathbf{A}$ of a graph $G = (V, E, L)$ is the binary matrix with entries $\mathbf{A}_{uv} | (u, v) \in E = 1$, $\mathbf{A}_{uv} | (u, v) \notin E = 0$.

The *degree matrix* $\mathbf{D}$ is the diagonal matrix with entries $\mathbf{D}_{uu} = d(u)$. The *Laplacian matrix* is $\mathbf{L} = \mathbf{D} - \mathbf{A}$. The *Laplacian spectrum* is the ordered set $\lambda_1 \leq \dots \leq \lambda_n$ of its $n = |V|$ eigenvalues. The Laplacian matrix is *symmetric*, as $\mathbf{A}_{ij} = \mathbf{A}_{ji}$ for any i, j , thus its eigenvalues are real-valued.

III. RELATED WORK

Subgraph matching features two main families of methods: *exact* methods (Section III-A) abide by subgraph isomorphism; *approximate* methods (Section III-B) relax subgraph isomorphism explicitly or implicitly, thus returning answers not isomorphic to the query. Table I presents the main familial characteristics.

A. Exact methods

Direct enumeration methods [61] exhaustively search for answers in the graph using *backtracking* techniques. These methods build on Ullmann's subgraph isomorphism algorithm [34] that leverages basic pruning rules, such as label- and degree-based pruning; VF2 [49] induces a traversal order on the nodes in the query; GADDI [6] computes matches of selected patterns in the k -neighborhood induced subgraphs of a set of sampled nodes. For a comprehensive evaluation of direct enumeration algorithms, see [61].

Filter and verification (FV) algorithms [32] follow a two-stage approach. First, in an offline stage, they build a *large* index of frequent structures. Then, in an online phase, at query time, they use the index to prune candidate nodes before performing direct enumeration. These methods mainly differ on the type of index they employ. Subgraphs: gIndex [25] indexes the most frequent subgraphs; GString [23] defines a grammar of basic structures to encode queries and graphs; SG-Match [24] mines frequent induced subgraphs called graphlets. Trees: TreePi [26] enumerates trees with a minimum support and stores them in a compressed index; Tree+ Δ [27] stores frequent trees along with discriminating edges; QuickSI [29] builds a tree index on edge and node labels to improve the node-enumeration order; Ntree [28] precomputes trees around each node's neighborhood. Paths: GraphGrep [62] enumerates paths to create a string representation of a subgraph; SPath [30] selects low-selectivity paths to improve the enumeration order; PathIndex [31] stores the labeled neighborhood of each node along edge-disjoint paths. Spectra: GCoding [47] computes an offline index with the largest eigenvalues of 2-hop neighborhoods and labeled neighborhoods and uses the interlacing theorem to prune candidates; PLGCoding [50] stores both paths and Laplacian spectra of nodes to improve the pruning power. This *lightweight* index is easy to maintain compared to structures such as subgraphs and trees; upon an update, eigenvalues need to be recalculated only for nodes within k hops of the update. In certain occasions [63], the eigenvalues can be recalculated on the fly. However, such spectral indexes are effective on queries comprising almost a whole graph in a collection. They relent on generic *subgraph* matching, whereby the query is typically much smaller than the full graph. Besides, their pruning capacity is limited to the initial *filtering* phase, while the spectrum is unused in the later *refinement* phase. We devise a method that surmounts these limitations in filtering and refinement and can be combined with any current or future enumeration technique, while we use the state-of-the-art enumeration of [43].

Preprocessing enumeration methods build a candidate space at query time to improve pruning power. These methods achieve state-of-the-art performance in subgraph matching [43]. GraphQL (GQL) [51] builds a candidate space on all possible query-node pairs passing an initial pruning, and then filters candidates using string matching over the label-neighborhoods of candidates before performing a pseudo-isomorphism test on the candidate space and a final subgraph isomorphism step. CECI [38] introduces *embedding clusters*, i.e., subgraphs of G that contain possible query answers with a common pivot node; then it uses an auxiliary tree structure [39] on the candidate space to store all possible candidate nodes and edges. DAF [48] induces a traversal order on the query to reduce candidates and refines the candidate space by dynamic programming and the *failing sets* method that prunes subgraphs not constituting part of a query answer. VEQ [37] improves the DAF filtering phase with neighborhood safety rules and its enumeration phase with static and dynamic

Table I: Algorithms’ comparison on whether they employ a large or small *offline index*, an *online structure*, exploit *high-order* information, apply to any dataset without requiring retraining, or return *exact* subgraph isomorphic answers.

Method family	Storage		Characteristics		
	Offline index	Online structure	High-order	Data-agnostic	Exact
Direct enumeration [6], [34], [49]	✗	✗	✗	✓	✓
Filter and Verification [23]–[31]	large	✗	✗	✓	✓
Spectral FV [47], [50]	small	✗	✓	✓	✓
Preprocessing enumeration [38], [39], [43], [48], [51]	small	✓	✗	✓	✓
Subgraph isomorphism relaxation [52]–[57]	large	✗	✓	✓	✗
Neural methods [58]–[60]	✗	✓	✓	✗	✗
PILOS	small	✓	✓	✓	✓

equivalences. RapidMatch (RM) [40] converts the query to a set of joins and processes results via relational operators.

Our work has most in common with preprocessing enumeration methods at the core, yet combines this core with an offline index-based initial filtering and with an *unprecedented* online spectral filtering that confers unparalleled pruning power.

B. Approximate methods

Approximate methods *relax subgraph isomorphism* to allow non-bijective mapping functions, train *neural models* to answer queries, or approximate solutions using *spectral techniques*.

Subgraph isomorphism relaxation compromises on subgraph isomorphism by allowing alternative node-to-node mapping functions. Homomorphism [52]–[54] allows injective functions to map edges to a path. Generalized subgraph search [55] allows specifying the *maximum path length* to map each query edge to the graph. Strong simulation [56] allows the mapping of one node to a group of nodes. NeMa [57] defines *neighborhood similarity* to account for label errors and structural differences among nodes.

Neural Methods learn low-dimensional node representations using expressive neural networks. Neural subgraph matching [60] learns vector representations of nodes whose k -hop neighborhoods contain the query. ISONET [59] improves interpretability by explicitly enforcing edge-to-edge mappings. DPGM [58] reduces the number of training examples by incorporating the graph’s geometry. Still, such methods require training, thus limit their generalizability to unseen data. We refer to [64] for an exhaustive discussion.

Spectral methods use the spectrum of a matrix representation of the graph to find *approximate* similar structures. The interlacing theorem (Theorem 1) finds use in answering similarity queries [63]. Graph spectra provide a versatile tool for detecting similar graphs [45] and finding node-to-node correspondences in the context of graph alignment [46], [65].

IV. ONLINE PILOS

Figure 2 shows the flowchart of PILOS. We create an index (2) that contains the largest S Laplacian Eigenvalues for every data graph vertex two-hop subgraph (1). When a query (3) arrives, we perform initial filtering (4) based on label, degree, and neighborhood information, using the index, and organize the candidate nodes passing this step in an auxiliary data structure, ADS (5). We further refine ADS (7) to remove false positives. Once we decide the ADS

is compact enough (6), we apply a backtracking algorithm (8) on ADS to return the matchings (9). PILOS differs from prior works, as *filter-and-verification* methods omit steps 5, 6, 7, *direct enumeration* omits steps 2, 5, 6, 7, while *preprocessing enumeration* discards the index (2). In the following, we describe the index (2), the filtering step (4), the ADS (5) and the refinements steps (7); then we give the complete solution and analyze its complexity. In a nutshell, we introduce a novel *online spectral filtering* approach and deploy it along with a curated selection of best-of-breed elements from prior work.

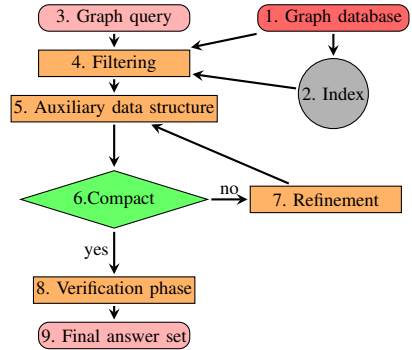


Figure 2: PILOS flowchart.

The enumeration we employ was proposed in [43]. A *matching order* is a permutation of V_q that determines the order by which we assess query vertices. We use the matching order of GQL [51], employing a left-deep join tree, to start from the query vertex having the fewest candidates, add it to the ordered list ϕ , and proceed to the neighbor query vertex $\notin \phi$ with fewest candidates. Denoting the set of nodes selected so far as μ , we first select a node from the candidate space of $C(\phi(0))$ and add its v_i to μ . In each step, we extend from the current v_i considering candidates that are neighbors of v_i and candidate nodes for the next query vertex in ϕ , and not already in μ , using an auxiliary data structure. We also use the *failing sets* method [48], which exploits previously acquired knowledge to prune unpromising paths; when backtracking from an unsuccessful partial result to select a new branch, we utilize information obtained during the previous failed traversal to prune invalid branches we encounter again. Lastly, we apply *dynamic equivalences* [37] to concurrently prune nodes that share the neighbors of a non-matching node.

A. Pruning Rules

It is possible to prune several nodes and edges in advance by simple *pruning rules* that characterize potential candidates

for each query node $q \in V_q$. A first such rule enforces *degree filtering*: a candidate node for a query vertex q should have the label and degree of q .

Rule 1 (LDF). A node u is an invalid candidate for a query vertex $q \in V_q$ if $L(u) \neq L_q(q)$ or $d(u) < d(q)$.

A second rule requires the label-neighborhood $\mathcal{N}_\Lambda(q)$ of query node q to be a subset of that of a valid candidate node.

Rule 2 (NLF). A node u is an invalid candidate for a query node $q \in V_q$ if $\mathcal{N}_\Lambda(q) \not\subseteq \mathcal{N}_\Lambda(u)$.

While LDF [34] manifests itself in all relevant previous works, NLF [66] has been utilized by recent *preprocessing-enumeration* algorithms. We now propose an additional, third rule that leverages the spectrum of the graph's Laplacian matrix. This rule results from the interlacing theorem [67], which states that the eigenvalues of a subgraph are always smaller than those of the graph.

Theorem 1 (Interlacing Theorem). Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a $n \times n$ real-valued symmetric matrix with spectrum $\alpha_1 \geq \dots \geq \alpha_n$ and $\mathbf{B} \in \mathbb{R}^{m \times m}$ with $m < n$ and spectrum $\beta_1 \geq \dots \geq \beta_m$ be a matrix obtained by deleting the i th row and column of $\mathbf{A}$. Then $\alpha_k \geq \beta_k \geq \alpha_{k+n-m}$ for $k = 1, \dots, m$.

Theorem 1 implies that the spectrum of the graph induced by the k -neighborhood of a query node q interlaces with that of a candidate node in $C(q)$. Since we are interested in *plain* (not induced) subgraphs, we drop the condition $\beta_k \geq \alpha_{k+n-m}$ and derive the following simplified rule:

Rule 3 (SF). A node u is an invalid candidate for query node $q \in V_q$ if $\lambda_t < \mu_t$ with $t \leq |\mathcal{N}_k(q)|$, where $\lambda_1 \geq \dots \geq \lambda_{|\mathcal{N}_k(u)|}$ is the spectrum of the subgraph $G[\mathcal{N}_k(u)]$ induced by $\mathcal{N}_k(u)$ and $\mu_1 \geq \dots \geq \mu_{|\mathcal{N}_k(q)|}$ is the spectrum of the subgraph $Q[\mathcal{N}_k(q)]$ induced by the k -neighborhood $\mathcal{N}_k(q)$.

As considering too many nodes would burden runtime, we use the 2-neighborhood $\mathcal{N}_2$ of query and candidate nodes.

As an example, consider the query and the graph in Figure 5e in the following. The spectra of q_1 and v_1 do not interlace, as $\lambda_3 < \mu_3$, hence v_1 is an invalid candidate for q_1 .

B. Offline index creation

The PILOS index stores the top S largest Laplacian eigenvalues of a k -hop neighborhood of each vertex in the data graph. Algorithm 1 shows the pseudocode for index creation. We extract the k -hop neighborhood $\mathcal{N}_k(u)$ of each vertex u by breadth-first search, store its Laplacian matrix $\mathbf{L}$, extract the top- S eigenvalues thereof, and store² them in the index. If the subgraph built starting from u has fewer than S eigenvalues, we pad the remaining ones with -1 . To reduce the time for index creation, if the k -hop subgraph of a node u leads to a subgraph of more nodes than a threshold,³ we set the eigenvalues of u to a max value; this measure incurs no

²We use $S = 30$ and $k = 2$.

³We set that threshold to 500.

significant loss of pruning power, as a node with such a dense neighborhood is unlikely to be pruned anyway, while it reduces the worst-case runtime for index creation.

Algorithm 1 Offline index creation

```

1: procedure OFFLINEINDEX( $G, S, k$ )
2:   for  $i = 0, \dots, N$  do
3:      $\mathbf{L} \leftarrow \text{LAPLACIAN}(G[\mathcal{N}_k(i)])$  ▷ Laplacian of  $k$ -hop BFS
4:      $\lambda_1, \dots, \lambda_S \leftarrow \text{Eigs}(\mathbf{L}, S)$  ▷ Top- $S$  eigenvalues from  $\mathbf{L}$ 
5:      $\text{EigD}[i] \leftarrow \lambda_1, \dots, \lambda_S$  ▷ EigD Eigen Data
   return EigD

```

C. Index-based filtering

When a query Q arrives, we extract the k -hop neighborhood Laplacian eigenvalues and keep only the top- S of those, indicated as $\text{Eig}Q$, for each vertex in Q , as we do for data vertices in the offline index (Algorithm 1, Section IV-B). To facilitate filtering, we create a query-based *online temporary index*, OTI , holding the following information for each query vertex:

- k -hop neighborhood Laplacian eigenvalues $\text{Eig}Q$;
- degree $d(q)$, i.e., number of neighbors;
- extended degree $\mathcal{ED}(q)$, i.e., number of k -hop neighbors;
- neighbor label count $\mathcal{N}(q, l) = |\{v \in \mathcal{N}_\Lambda(q) | L_q((q, v)) = l\}|$, i.e., number of neighbors having label l ;
- extended k -hop label count $\mathcal{EN}(q)$, i.e., number of k -hop neighbors having each label.

Filtering (this section) uses $\text{Eig}Q$, $d(q)$, and $\mathcal{N}(q, l)$ for each label l ; initial refinement (Section IV-E) uses $d(q)$; simple refinement (Section IV-F) uses $\mathcal{N}(q, l)$ and $d(q)$; eventually, advanced refinement (Section IV-G) uses $\text{Eig}Q$, $\mathcal{EN}(q)$, and $\mathcal{ED}(q)$.

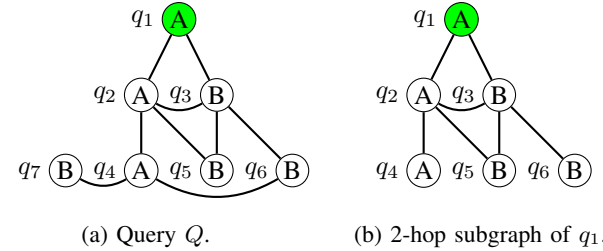


Figure 3: OTI creation; hop-1: $d(q_1)=2$, $\mathcal{N}(q_1, A)=1$, $\mathcal{N}(q_1, B)=1$; hop-2: $\mathcal{ED}(q_1)=5$, $\mathcal{EN}(q_1, A)=2$, $\mathcal{EN}(q_1, B)=3$.

Figure 3 illustrates how we create OTI with an example. **Figure 3a** shows an example query graph of seven nodes, $q_1, \dots, q_7$, each having one of two labels, $\{A, B\}$. **Figure 3b** shows the 2-hop subgraph of node q_1 (colored in green), which results from a 2-hop *breadth-first search* (BFS) exploration. Starting from q_1 , the first BFS hop discovers q_2 and q_3 , while the second hop arrives at q_4 , q_5 , and q_6 . As **Figure 3b** shows, even though the BFS reaches q_4 and q_6 , it falls short of the edge between them, which would be discovered in a third hop. OTI maintains the following information on q_1 : as q_1 has two neighbors, q_2 and q_3 , $d(q_1) = 2$; since q_2 has label A and q_3 has label B , $\mathcal{N}(q_1, A) = 1$ and $\mathcal{N}(q_1, B) = 1$; the subgraph extracted by a two-hop exploration, excluding q_1 ,

is $[q_2, q_3, q_4, q_5, q_6]$, $\mathcal{ED}(q_1)=5$; lastly, the labels in this two-hop subgraph of q_1 are $\mathcal{EN}(q_1, A) = 2$ due to nodes $[q_2, q_4]$, and $\mathcal{EN}(q_1, B) = 3$, due to nodes $[q_3, q_5, q_6]$. We complete *OTI* by repeating this operation for all query nodes.

Algorithm 2 presents our index-based filtering, which applies the safety rules of Section IV-A to yield an initial *candidate space CS* of candidate data graph nodes for each query vertex q . To efficiently find such candidates, we store a temporary *reverse index* (RI) that provides the data graph nodes having each query label (Line 3). Then, for each query vertex q (Line 4), we iterate only through data graph nodes v having the same label as q , i.e., $L(v) = L(q)$, using the reverse index (Line 5); this label-based selection forms the label part of LDF (Line 6); we also check degrees to apply the degree part of the same LDF rule (Line 7). Next, we apply SF (Line 8), i.e., check whether data eigenvalues $EigD$ are larger than query eigenvalues $EigQ$. For a data graph node v passing the above tests, we extract its neighborhood and check whether it contains the neighborhood of query vertex q , i.e., conduct NLF (Line 9). Thus, we prioritize the low-cost LDF and SF over the more costly NLF.

Eventually, we add the data graph nodes that pass all filtering steps for each query vertex q_i to a *candidate node set* $C(q_i)$ (Line 10). Henceforward, we refer to a data graph node v_j being a *candidate node* for the position of query vertex q_i as $C(q_i, v_j)$. For example, $C(q_1, v_{10})$ refers to data graph node v_{10} as a *candidate node* for the position of query vertex q_1 .

Algorithm 2 Filter

```

1: procedure FILTER( $CS, G, EigD, OTI$ )
2:   for  $i = 0 \dots \Sigma_q$  do                                ▷ iterate over query labels
3:      $RI \leftarrow G(L(i))$                                 ▷ reverse index of data nodes for label
4:   for  $i = 0 \dots N_q$  do                                ▷ iterate over query nodes
5:      $N_\ell \leftarrow RI(L(i))$                                 ▷ data nodes having query node label
6:     for  $j = 0, \dots, |N_\ell|$  do                            ▷ loop over nodes in  $N_\ell$ 
7:       if  $d(q_i) \geq d(v_j)$  then
8:         if SF( $EigD[j], OTI.EigQ[i]$ ) then                ▷ spectral filter
9:           if NLF( $G, OTI, j, i$ ) then                    ▷ label filter
10:             $CS[i].add(j)$                                 ▷ add  $j$  to  $q_i$ 's cand. space

```

D. Auxiliary Data Structure (ADS)

The index-based filtering of Section IV-C (Algorithm 2) populates the candidate space CS with candidate nodes that pass all filtering steps, arbitrarily ordered by ID; we maintain this order throughout the algorithm. Next, we build an *auxiliary data structure*, *ADS*, resembling those in CECI [38] and DAF [48], by adding edges among candidate nodes in CS according to the following rule:

Rule 4. Given candidate nodes $C(q_i, v_j), C(q_{i'}, v_{j'}) \in CS$, with $q_i, q_{i'} \in V_q$, and $v_j, v_{j'} \in V$, we add edge $(C(q_i, v_j), C(q_{i'}, v_{j'}))$ to *ADS* if and only if there are edges $(i, i') \in E_q$ and $(j, j') \in E$.

The resulting *ADS* structure encapsulates all possible matches in the data graph, hence we may perform pruning and isomorphic tests directly on *ADS*.

Figure 4a shows an example query graph Q with 5 nodes, which is issued on the 10-node data graph G of Figure 4b pro-

ducing the candidate space CS of Figure 4c, featuring a *candidate node set* for each query vertex. For instance, candidate nodes for the position of query vertex q_1 have the same label as q_1 , $L(q_1) = A$, i.e., belong to set $\{v_1, v_2, v_4, v_7, v_9, v_{10}\}$, and at least equal degree as q_1 , $d(q_1) = 3$, i.e., belong to set $\{v_1, v_5, v_4, v_8, v_{10}\}$. The intersection of these two sets yields the *candidate node set* $C(q_1) = \{v_1, v_4, v_{10}\}$. We repeat this operation over each query vertex to obtain its candidate node set. Figure 4c shows the *ADS* generated by issuing Q on G with LDF, including the edges added by Rule 4.

To add edges, for instance, to $C(q_1, v_4)$, we first inspect all neighbors of q_1 in the query graph (Figure 4a), i.e., q_2, q_3, q_5 ; we infer that we should add edges to candidates that belong to the respective candidate sets, $C(q_2), C(q_3), C(q_5)$. Next, we check the neighbors of v_4 in the data graph (4b), i.e., v_1, v_2, v_6 ; we deduce that we have to add edges to those candidates whose sets have a neighbor of v_4 in their candidate sets. For instance, candidate set $C(q_2)$ includes nodes v_1, v_2 , thus we add edges $(C(q_1, v_4), C(q_2, v_1))$ and $(C(q_1, v_4), C(q_2, v_2))$. Similarly, candidate set $C(q_3)$ includes node v_1 , hence we add edge $(C(q_1, v_4), C(q_3, v_1))$, while candidate set $C(q_5)$ features node v_6 , so we add edge $(C(q_1, v_4), C(q_5, v_6))$. We store these edges as adjacency lists for each candidate node in *ADS*, representing each node by a pair of unsigned integers (q_{id}, v_{id}) . For example, in 4c, the edges added to node $C(q_1, v_{10})$, indicated in green color, are stored as a set of pairs, $\{(2, 7), (2, 9), (5, 5), (5, 8)\}$. The ensuing *ADS* captures all necessary information about candidate nodes, thus we dispense of the data graph after creating the *ADS*. We perform the following steps of refinement, ordering, and enumeration directly on *ADS*.

E. Initial refinement by relaxed degree

Given the way we add edges to *ADS*, the total number of edges, or *relaxed degree* ($\mathcal{RD}$), of a candidate node in *ADS* should at least equal the respective query node degree $d(q)$, hence the rule:

Rule 5 (Relaxed Degree safety). $\mathcal{RD}(v) \geq d(q), v \in C(q)$

After adding edges to *ADS*, we check whether candidate nodes therein pass the relaxed degree criterion. To visualize Rule 5, consider node q_1 in Figure 4a with degree $d(q_1) = 3$ and the respective candidate node $C(q_1, v_{10})$ in Figure 4c with degree $d(C(q_1, v_{10})) = 4$, due to edges to $C(q_2, v_7), C(q_2, v_9), C(q_5, v_5)$ and $C(q_5, v_8)$ (all colored in green). Since $4 \geq 3$, candidate node $C(q_1, v_{10})$ passes the relaxed degree check. On the other hand, in the case of candidate node $C(q_5, v_8)$, it is $d(q_5) = 2$ and $d(C(q_5, v_8)) = 1$, hence $C(q_5, v_8)$ fails the relaxed degree check as a candidate for q_5 . While $C(q_5, v_8)$ passes the degree check (LDF) in the data graph, due to also having neighbors v_7, v_9 (Figure 4b), those nodes do not make valid candidates for q_1 or q_4 , hence invalidate $C(q_5, v_8)$.

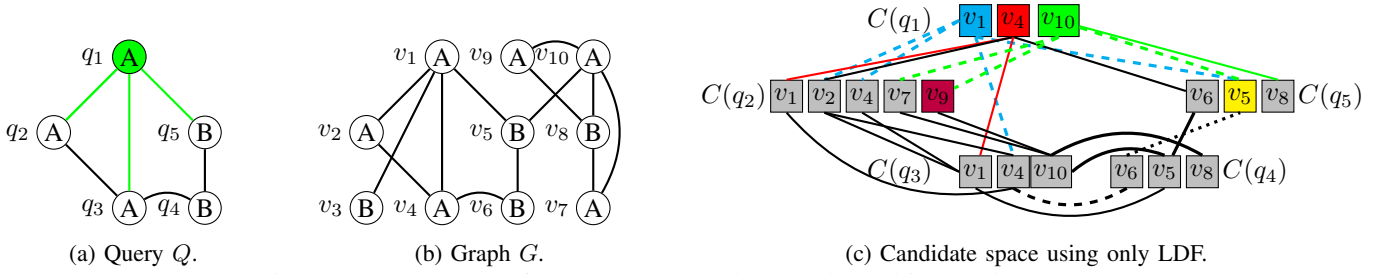


Figure 4: An example of a query graph, a data graphs and its candidate space

F. Simple refinement

After disqualifying candidate nodes by the $\mathcal{RD}$ criterion, we check whether the neighborhood of each remaining candidate node u is correct using three complementary rules for neighbor evaluation. First, we remove edges by the following rule.

Rule 6 (Edge Safety). *If there is a single edge from a candidate node $C(q, v)$ to a candidate node set $C(q_i)$, linking $C(q, v)$ to candidate node $C(q_i, v_j)$, then there should be no duplicate edge from $C(q, v)$ to data graph node v_j in any other candidate node set $C(q_o)$, with $q \neq q_o$.*

For instance, in **Figure 4c**, candidate node $C(q_1, v_4)$ has a *single* edge (colored red) to candidate node set $C(q_3)$, linking $C(q_1, v_4)$ to candidate node $C(q_3, v_1)$. Still, there is also a *duplicate* edge from $C(q_1, v_4)$ to $C(q_2, v_1)$, i.e., data graph node v_1 in candidate node set $C(q_2)$; thus, edge $(C(q_1, v_4), C(q_2, v_1))$ is removed by Rule 6. While we remove the edge to $C(q_2, v_1)$, we retain the one to $C(q_3, v_1)$, since there is also another edge from $C(q_1, v_4)$ to candidate node set $C(q_2)$, linking $C(q_1, v_4)$ to candidate node $C(q_2, v_2)$, thereby preserving v_4 as a valid candidate for the position of query vertex q_1 . If there were only a *single* edge from $C(q_1, v_4)$ to candidate node set $C(q_2)$, which linked $C(q_1, v_4)$ to a *duplicate* data graph node v_1 , and which we would thus remove by Rule 6, then we would also safely remove $C(q_1, v_4)$ from the candidate set altogether, as it would then retain an insufficient degree to match query vertex q_1 . The following rule formalizes this principle.

Rule 7 (Real Degree safety). *A candidate node $C(q, v)$ should have at least as many neighbor candidate node sets as the neighbors of query vertex q in the query graph.*

We call the count of neighbor candidate node sets **real degree**, to distinguish it from the *relaxed degree* of Rule 5, which may count more than one edge per candidate node set (i.e., query vertex). By construction, we add edges between candidate nodes only if an edge exists between their respective query vertices. Thus, by the pigeonhole principle, a candidate node $C(q, v)$ that satisfies Rule 7 has the correct *set* of neighbor candidate node sets. In reverse, we safely prune any candidate node that has fewer neighbor candidate node sets than the degree of its respective query vertex. In **Figure 4c**, candidate node $C(q_1, v_{10})$ (colored green), for instance, has edges to $C(q_2)$ and $C(q_5)$, hence its *real degree* of is 2;

as the respective query vertex q_1 has degree 3, we safely prune $C(q_1, v_{10})$.

To apply Rule 7 efficiently, we store, for each candidate node, a *neighbor count* (NC) array with the count of its edges to each candidate node set. For instance, the NC array for $C(q_1, v_{10})$ is $\{0, 2, 0, 0, 2\}$, where the i^{th} array entry corresponds to q_i ; as the array has two non-zero entries, the real degree of $C(q_1, v_{10})$ is 2. When we prune the edge to a neighbor candidate node, or a neighbor candidate node itself, we reduce the respective array entry; should any such entry drop to zero, Rule 7 fails hence we prune the candidate node.

Lastly, we apply NLF by the following rule, also used in [37], which checks whether a candidate node matches the neighborhood labels of its respective query vertex.

Rule 8 (Neighborhood Label safety). *A candidate node $C(q, v)$ should have $\forall \ell \in \mathcal{N}_\Lambda(q), \mathcal{N}(C(q, v), \ell) \geq \mathcal{N}(q, \ell)$.*

When counting neighborhood labels for a candidate node, we allow counting multiple vertices of the same q_{id} , provided they have different v_{id} . For instance, to check the neighborhood of candidate node $C(q_1, v_1)$ (colored blue) in **Figure 4c**, we first count neighbors in $C(q_2)$, and find $\{v_2, v_4\}$, both of label A , hence $\mathcal{N}(C(q_1, v_1), A) = 2$; we then check $C(q_3)$, where we find v_4 as the only neighbor. As we have already⁴ counted v_4 , we do not increase the count. Lastly, we check $C(q_5)$, where v_5 is a neighbor, hence $\mathcal{N}(C(q_1, v_1), B) = 1$.

Building on the example of candidate node $C(q_1, v_{10})$ in **Figure 4c** (colored green), the respective query vertex q_1 has label count $\mathcal{N}(q_1, A) = 2$ and $\mathcal{N}(q_1, B) = 1$, while $\mathcal{N}(C(q_1, v_{10}), A) = 2$ ($\{v_7, v_9\}$) and $\mathcal{N}(C(q_1, v_{10}), B) = 2$ ($\{v_5, v_8\}$), thus $C(q_1, v_{10})$ would qualify by Rule 8, yet as we saw it fails by Rule 7. For another example, the respective query node q_2 of $C(q_2, v_9)$ has degree 2 with neighbors $\{q_1, q_3\}$ and label count $\mathcal{N}(q_2, A) = 2$ since both neighbors have label A . Likewise, $C(q_2, v_9)$ (colored purple) has real degree 2 with neighbors $\{q_1, q_3\}$, hence it is degree-safe; still, it is not neighborhood-label-safe since $\mathcal{N}(C(q_2, v_9), A) = 1$, as q_{10} is a duplicate neighbor; due to the same duplicate neighbor, $C(q_2, v_9)$ fails the edge safety test (Rule 6) too.

⁴We use an unordered set for each candidate node to check for v_{id} uniqueness.

G. Advanced refinement

We augment the simple refinement of Section IV-F with more advanced pruning steps. To perform this advanced refinement, we first extract the 2-hop neighborhood of a candidate node $C(q, v)$ by BFS, calculate the top S eigenvalues⁵ of its Laplacian matrix, and compare them to those of the query vertex q by Rule 3, as we do for the offline index (Section IV-B) and the online temporary index (Section IV-C). Nevertheless, contrastingly to those cases, in the BFS on ADS , we may encounter duplicate data graph nodes, i.e., nodes having the same v_{id} but different q_{id} . We consider such duplicates as the same node and merge their instances to the same v_{id} , as they are the same node in the data graph. Still, in the first BFS hop, we add such a duplicate to the BFS queue with its q_{id} , as pair (q_{id}, v_{id}) , so that subsequently, in the second BFS hop, we extract the distinctive neighbors of each duplicate. To avoid redundant computations, we eschew exploring nodes that have the same v_{id} or q_{id} as the node under evaluation.

1) *Extended neighborhood safety*: We add two additional safety rules, extending the degree and neighborhood safety rules of Section IV-F from one hop to k hops. We instantiate them with $k = 2$. The first rule extends the degree safety rule (Rule 7). Extending the concept of *real degree* to k hops, we get the number of k -hop neighbors of a query vertex, which should be matched by that of a candidate node. The signification *real* remains as in Section IV-F, i.e., we count distinct candidate node sets returned by the BFS exploration.

Rule 9 (Extended Real Degree safety). *A candidate node $C(q, v)$ should have as many k -hop neighbor candidate node sets as the k -hop neighbors of query vertex q .*

The next rule extends the neighborhood label safety rule (Rule 8), by checking whether a candidate node matches the labels of the respective query vertex in its k -hop neighborhood.

Rule 10 (Extended Label Neighborhood safety). *A candidate node $C(q, v)$ should have $\forall \ell \in \mathcal{EN}_\Lambda(q), \mathcal{EN}(C(q, v), \ell) \geq \mathcal{EN}(q, \ell)$.*

We calculate the extended real degree and extended (k -hop) label neighborhood in the OTI , by the same rules as in Section IV-C. For instance, the k -hop neighborhood of candidate node $C(q_5, v_5)$ (colored yellow) in **Figure 4c** is indicated with dashed lines. Notably, while $C(q_1, v_{10})$ is a neighbor of $C(q_5, v_5)$, its edge to 2nd-hop neighbor $C(q_5, v_8)$ has no dashed line, because $C(q_5, v_8)$ has the same q_{id} as the evaluated node, $C(q_5, v_5)$; we consider only one candidate node at a time for a specific query vertex. The extended degree is $\mathcal{ED}(C(q_5, v_5)) = 4$, as $C(q_5, v_5)$ has 2-hop neighbors $\{q_1, q_2, q_3, q_4\}$, while its extended 2-hop label neighborhood is $\mathcal{EN}(C(q_5, v_5), A) = 6$ due to 2-hop neighbors $\{v_1, v_2, v_4, v_7, v_9, v_{10}\}$ and $\mathcal{EN}(C(q_5, v_5), B) = 1$ due to neighbor v_6 .

⁵We may adjust the number of eigenvalues S based on the query size; however, we keep the same S throughout the algorithm for simplicity.

2) *Online Laplacian eigenvalue pruning*: On each candidate node, after applying the extended neighborhood safety rules, we proceed to complete the refined k -hop pruning with online spectral filtering based on Laplacian eigenvalues as Algorithm 3 indicates. We first collect the candidate node's k -hop neighborhood and evaluate it by extended neighborhood safety (Line 5), while keeping information about encountered edges in a Laplacian matrix $\mathbf{L}$. If the candidate passes that check, then we finalize the Laplacian matrix of that k -hop neighborhood (Line 8), calculate its top- S eigenvalues (Line 9), and perform online Laplacian eigenvalue pruning (Line 10); if the node passes this filter, we consider it a valid candidate. We set an invalidated candidate node as invalid (Lines 6 and 11), so that subsequently examined candidate nodes are up-to-date regarding the change. To reduce the eigenvalue calculations, we ban computing the eigenvalues for candidate nodes corresponding to query leaf vertices. Thus, a candidate node $C(q_i, v_j)$ registers the deletion of another candidate $C(q_{i'}, v_{j'})$ if $i > i'$, or $i = i'$ and $j > j'$. For the sake of efficiency, we ignore invalidated nodes and their edges during an iteration and collectively remove them from ADS at the end of each iteration.

Algorithm 3 Refined k -hop

```

1: procedure REFINED  $k$ -HOP( $OTI, ADS$ )
2:   for  $i = 0, \dots, N_q$  do                                ▷ loop over query nodes
3:     for  $j = 0, \dots, |C(q_i)|$  do                          ▷ loop over nodes in  $C(q_i)$ 
4:        $\mathbf{L} \leftarrow \text{null}$                                      ▷ empty Laplacian
5:       if  $\neg \text{ENS}(ADS[i][j], OTI, \mathbf{L})$  then                 ▷ extended neighborhood safety
6:          $ADS[i][j].\text{valid} \leftarrow \text{false}$ 
7:         continue
8:        $\mathbf{L} \leftarrow \text{FINALIZE}(\mathbf{L})$                            ▷ finalize  $k$ -hop neighborhood Laplacian
9:        $EigD \leftarrow \text{EIGS}(\mathbf{L}, S)$                          ▷ Top- $S$  eigenvalues
10:      if  $\neg \text{SF}(EigD[j], OTI.EigQ[i])$  then
11:         $ADS[i][j].\text{valid} \leftarrow \text{false}$ 

```

Figure 5 shows an example of Online Laplacian Eigenvalue pruning with the 4-clique query graph Figure 5a, query node q_1 , and candidate node v_1 . Previous rules fail to detect that v_1 , which does not form a triangle with its neighbors as q_1 does, is not a candidate for q_1 . By spectral pruning, we extract the 2-hop neighborhood of v_1 (Figure 5b) in the candidate graph, compute its Laplacian matrix (Figure 5d) and its eigenvalues $\lambda = [4.73, 4, 3.41, 1.26]$ for v_1 , store them in descending order, and compare them to those for q_1 in OTI , $\mu = [4, 4, 4, 0]$ (Figure 5e). By Rule SF, if $\mu_i > \lambda_i$ for some i , we prune v_1 ; as $\mu_3 = 4.0 > 3.41 = \lambda_3$ satisfies this condition, we prune the node without checking other eigenvalues.

H. Complete algorithm

Algorithm 4 presents the PILOS algorithm. When the query arrives, we create the online temporary index (Line 2) that contains, for each query vertex, its k -hop neighborhood eigenvalues and additional information required for the pruning rules. Next, we perform index-based filtering (Line 3), given the spectral index and store the unfiltered nodes in the auxiliary data structure. Next, we complete the auxiliary data structure (Line 4) by adding edges between candidate nodes (Section IV-D). We then proceed to refined filtering on ADS . First, we perform *initial refinement* (Line 6) via

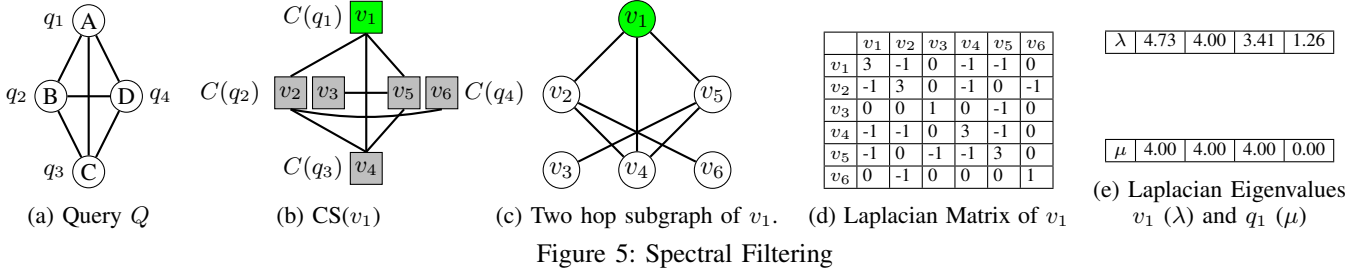


Figure 5: Spectral Filtering

a quick scan of the nodes based on the relaxed degree rule; we reiterate until there is no change, hence the process *converges* (Line 5). Upon convergence, we perform *simple refinement* (Line 8), checking the neighborhood of candidate nodes. Next, *advanced refinement* (Line 10), in which we extend the simple refinement to the two-hop neighborhood with additional spectral filtering using Laplacian eigenvalues. After each iteration of advanced refinement, we repeat simple refinement internally to remove the nodes that fail the one-hop neighbor test (Line 12). Eventually, we pass surviving candidates to a *verification* algorithm (Line 13).

Algorithm 4 PILOS algorithm

```

1: procedure PILOS( $G, Q, EigD$ )
2:    $OTI \leftarrow \text{QUERY\_INDEX}(Q)$   $\triangleright$  calculate online temporary index
3:    $ADS \leftarrow \text{FILTER}(ADS, G, EigD, OTI)$   $\triangleright$  filter with LDF, NLF, SF
4:    $ADS \leftarrow \text{ADD\_EDGES}(G, Q, ADS)$   $\triangleright$  complete  $ADS$  with Edges
5:   while not converged do
6:     INITIAL REFINEMENT( $ADS, OTI$ )
7:   while not converged do
8:     SIMPLE REFINEMENT( $ADS, OTI$ )
9:   while not converged  $\wedge$   $counter \leq \omega$  do
10:    REFINED  $k$ -HOP( $ADS, OTI$ )
11:   while not converged do
12:    SIMPLE REFINEMENT( $ADS, OTI$ )
13:   VERIFICATION( $ADS$ )  $\triangleright$  backtracking on  $ADS$ 

```

We repeat the first two refinement procedures until they bring no further improvement, whereas prior works delimit repetitions for efficiency. For each candidate node in ADS , we use a Boolean value that indicates whether it has to be evaluated, initialized as true and set to false after evaluation. When we prune a candidate, we remove all edges to its neighbors in the candidate space and set their check value to true to re-evaluate those neighbors in the next refinement iteration. In advanced refinement, we stop the procedure after ω iterations⁶ if not converged; in addition, if a candidate node's k -hop subgraph exceeds a threshold α , we stop the evaluation and consider the candidate valid; this measure confers efficiency, as the node may be pruned later in a less costly fashion.

On data sets having few labels, we complement α with a threshold β on the total number of nodes visited during the second-hop exploration, as the BFS traversal may then encounter many duplicate nodes which increase its stack and burden the computations to calculate the k -hop subgraph.

The rationale for these thresholds is twofold. A larger data node subgraph (i) is less likely to be pruned by the eigenvalue and extended safety rules, and (ii) needs more time needs to extract the subgraph and calculate eigenvalues, hence becomes a bottleneck to total query evaluation time.

⁶We set $\omega = 3$ in our implementation.

I. Complexity Analysis

The *offline index* requires $\mathcal{O}(NS)$ space where N is the number of nodes and S the number of top eigenvalues used. The time complexity to build it is $\mathcal{O}(N^4)$, considering cubic time for the eigen-decomposition at every node. However, we limit N to a small constant α^7 at every node, hence complexity drops to $\mathcal{O}(N\alpha^3)$. Table II presents the index construction time and memory needs with $S = 30$ and $\alpha = 500$.

Table II: Index construction time and memory

Index Construction	Dataset					
	WN	DB	AM	EU	YT	UP
Memory (MB)	9	36	46	99	130	432
Runtime (Sec)	92	387	1212	524	2019	8679

ADS takes $\mathcal{O}(|E| \times |E_q|)$ space, as earlier auxiliary data structures for preprocessing enumeration; it achieves the maximum size of its vertex set when the data graph has only one label, $|\Sigma| = 1$, and all query vertices have the same degree, hence the candidate space size is $|ADS| = |V| \cdot |V_q|$. In such a degenerate case, index-based filtering iterates through all query vertices and all data nodes to evaluate LDF, NLF and SF in $\mathcal{O}(|V||V_q|)$ time, ignoring small constants like the numbers of eigenvalues and labels. Likewise, initial refinement takes $\mathcal{O}(|V||V_q|)$ time to check all candidate nodes. In simple refinement, we check each candidate node's neighborhood. For Rule 6 and Rule 7, we extract relevant information from the NC array maintaining $\mathcal{O}(|ADS|)$ complexity. To check candidate node uniqueness and sum up unique node labels for Rule 8, we iterate over all neighbors of each candidate node in $\mathcal{O}(|ADS| \max_d)$, where $\max_d$ is the maximum degree. For advanced refinement, we perform a k -hop BFS to create the Laplacian matrix, while the calculation of Laplacian eigenvalues is cubic. As we bound the maximum k -hop subgraph size with the α threshold, the complexity is $\mathcal{O}(|ADS|(\alpha + |E|))$ for BFS and $\mathcal{O}(|ADS|\alpha^3)$ for the eigen-decomposition.

V. EXPERIMENTS

We run experiments on an Intel Core i9-10940X, 378GB machine @3.30GHz running Linux 4.15.0. We implemented⁸ all algorithms in C++, extending the framework of [43]; for Laplacian eigenvalues we use the C++ *eigen* library [68].

Baselines. We compare PILOS to VEQ [37], RM [40] and the enhanced versions of GQL [51], DAF [48], and CFL [39] proposed in [43]. Enhancements include an ordering method

⁷We set $\alpha = 500$ in the offline index and $\alpha \in [25, 125]$ in online filtering.

⁸Code: <https://github.com/constantinosktsas/Pilos-Subgraph-Matching>.

in GQL, a candidate space with the set-of-intersection enumeration in DAF [38], [43], [48] with failing sets [48] enabled, improving performance on queries of more than 16 nodes; we incorporated such enhancements into PILOS in Section IV.

Datasets. We conduct experiments on six datasets exhibiting diverse characteristics in terms of number of nodes, labels, and average degree. Table III summarizes data characteristics.

- Wordnet (WN) [69], a graph of word semantic relationships.
- DBLP (DB) [70], a co-authorship collaboration network.
- Amazon (AM) [71], Amazon product co-purchasing network
- eu2005 (EU) [72], [73], a crawl of linked .eu pages.
- YouTube (YT) [70], a network among YouTube users.
- US Patents (UP) [74], a large Patent-citation network.

Table III: Data characteristics; number of nodes $|V|$; number of edges $|E|$; number of labels $|\Sigma|$; average degree $\text{avg } d$.

Dataset	Abbr.	$ V $	$ E $	$ \Sigma $	$\text{avg } d$
WordNet	WN	76 853	120 399	5	3.1
DBLP	DB	317 080	1 049 866	15	6.6
Amazon	AM	401 692	2 198 486	6	10.9
eu2005	EU	862 664	16 138 468	40	37.4
Youtube	YT	1 134 890	2 987 624	25	5.3
US Patents	UP	3 774 768	16 518 947	20	8.8

Due to page limit, we exclude some datasets from the results besides query time in Figure 10, Table IV and Table V.

Query generation. We generate queries of size $\{32, 64, 96, 128, 256\}$; as in previous works [43], [75]–[78], we randomly select a node and perform a random walk to collect the specified amount of nodes; if the walk does not reach new nodes for ten consecutive steps, it chooses a node among visited ones and continues therefrom; if it does not discover new nodes for thirty consecutive steps, it returns to the initial node and chooses an unvisited neighbor to continue. After we collect the desired nodes we add all edges between them to extract an *induced* connected subgraph of the data graph. We generate 400 queries for each size, 2 000 queries in total.

Termination criterion. As a subgraph matching query may return an overwhelming number of results or no answer within several minutes, we report, for each query, the time to return up to 100 000 results; if this task does not end in 10 minutes, we mark the query as *unresolved* and record its elapsed time as 10 minutes by convention. In Section V-D we also report results with a 60-minute time limit.

Hyperparameters. On WN and AM, which have few labels, we set the number of nodes visited to $\beta=500$ (Section IV-H) and ignore β on other datasets. We set $\alpha = 25$ on WN, AM, YT and EU, $\alpha = 75$ on UP, and $\alpha = 125$ on DB. We compute the top-30 Laplacian eigenvalues of each node’s 2-hop neighborhood. We chose a 2-hop neighborhood as prior work [57], [79] has shown that a larger k -hop provides an insufficient pruning advantage at a large runtime cost.

A. Index Pruning Power

Here we assess the pruning power gained when adding *Spectral Filtering* to *Neighborhood Label Filtering* with LDF (NLF), to get (NLFS). **Figure 6** shows the results in a

dual-axis plot; the bars marked on the left y -axis show the number of remaining candidates after applying NLF and NLFS; the line marked on the secondary right y -axis shows the percentage of improvement that NLFS brings NLF. For all labeled data, plain NLF removes the majority of false positive candidate nodes, as they leverage labels that help discriminate among nodes. Nevertheless, the pruning power of NLF is compromised when we reduce the number of labels, as then NLF cannot distinguish nodes based on labels; in data sets having only one label, NLF distinguished nodes only by their degree. In these scenarios, the additional spectral information advances pruning power. All data sets and query sizes exhibit an improvement using NLFS over NLF. However, the improvement with UP is less than 1%, while with DB it is around 1.5% and with YT 3.5%. As expected, with the WN dataset, which has five labels in total and around 82% of the vertices have the same label, the pruning power of NLF is severely limited. The pruning advantage gained by NLFS is around 13% for query size 32 and rises with query size to up to 17% for query size 256. The difference in the number of candidate nodes is around 133 000 more candidates with query size 32 and 623 500 with query size 256. Thus, extending NLF to NLFS is particularly beneficial for data with fewer labels.

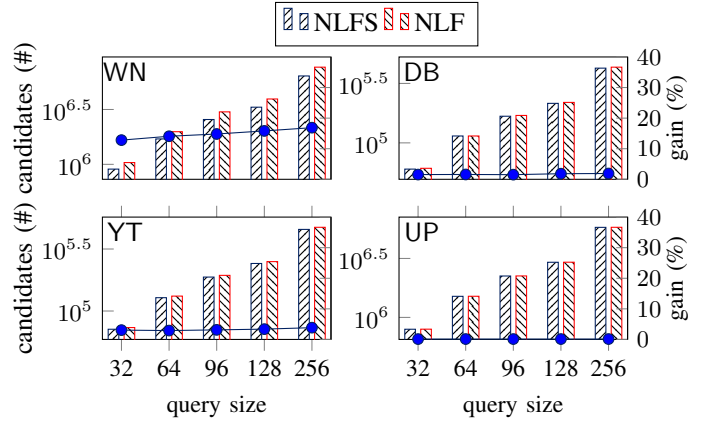


Figure 6: NLF and NLFS pruning power; bars show numbers of candidates, line plots pruning improvement with NLFS.

Even while PILOS prunes candidate nodes by spectral filtering in the online phase, it also benefits from using the offline index on data sets with fewer labels, measurably reducing initial candidate nodes and hence the time to evaluate them. Besides, as advanced refinement only runs for a few iterations, the offline index usage enables the algorithm to prune more candidate nodes in that stage, which would have otherwise required further iterations. Therefore, a structural index would also benefit queries with *wildcard*, i.e., unspecified, labels.

B. Filtering-Refinement Steps Evaluation

Here, we evaluate Index-based filtering (F) (Section IV-C), Simple Refinement (SR) (Section IV-F) and Advanced Refinement (AR) (Section IV-G) individually. Figure 7 shows that relying solely on Index-based filtering is inadequate for effective pruning. Simple Refinement reduces the number of candidates by up to two additional orders of magnitude

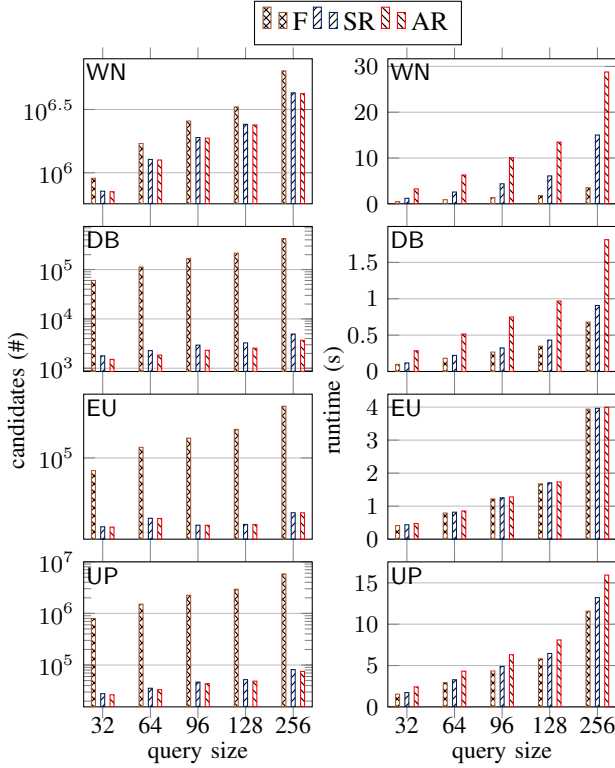


Figure 7: Avg pruning power (left) & time (right) for Filtering (F), Simple Refinement (SR), Advanced Refinement (AR).

while incurring only a negligible time cost. Advanced Refinement, which leverages both 2-hop neighborhood information and Laplacian eigenvalues, eliminates candidates that simpler pruning techniques fail to detect. The improvement using AR over SR is about 27% for DB, 7% for UP, 1% for YT and WN, and 0.5% for EU. Removing more challenging candidates takes more runtime, hence AR incurs the highest runtime.

C. Refinement evaluation

Here, we present results on pruning power and filtering time. **Figure 8** shows how many candidates each method leaves after filtering and the pruning gain PILOS confers over them as a percentage, while **Figure 9** shows the time for filtering, on all data sets. PILOS retains a competitive pruning advantage over prior work, even the most recent VEQ. Remarkably, PILOS outperforms VEQ by at least 10%, GQL by 5%, and RM by 10% within WN, which contains larger candidate spaces and exhibits a skewed edge-label distribution, despite comprising only 70 000 nodes. The gain relative to DAF and CFL grows from 13% on small queries to 24% on large queries. This additional pruning affects the pruning time by 3sec on query size 32 and 29sec on query size 256, while others complete the pruning within 1sec for small queries and 2sec for the largest queries. This latency is worthwhile for the sake of reducing the query response time (Section V-D).

The DB data set yields a smaller candidate space than others, yet it presents pruning opportunities due to its structural diversification. PILOS outstrips other algorithms with 20–80% higher pruning power across query sizes. For instance, on

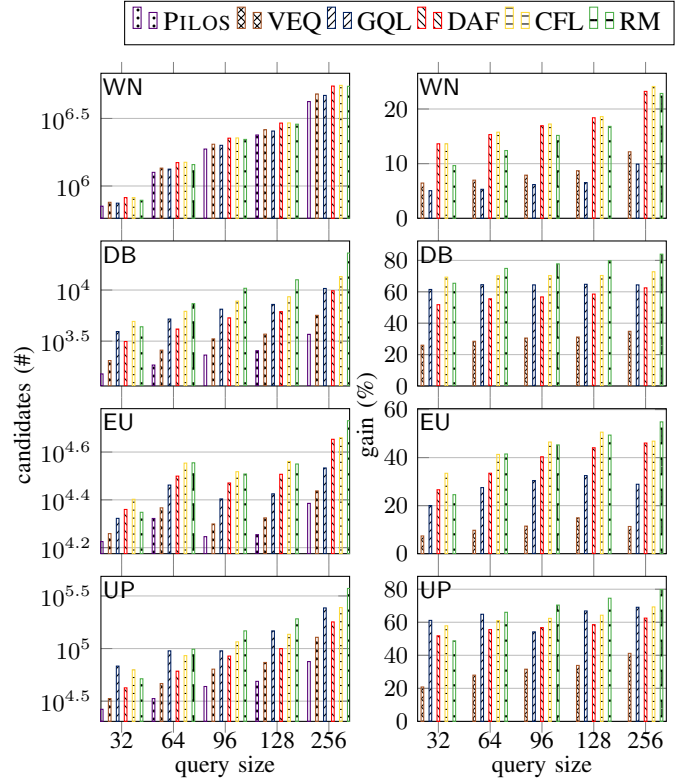


Figure 8: Avg pruning power (left); PILOS pruning gain (right).

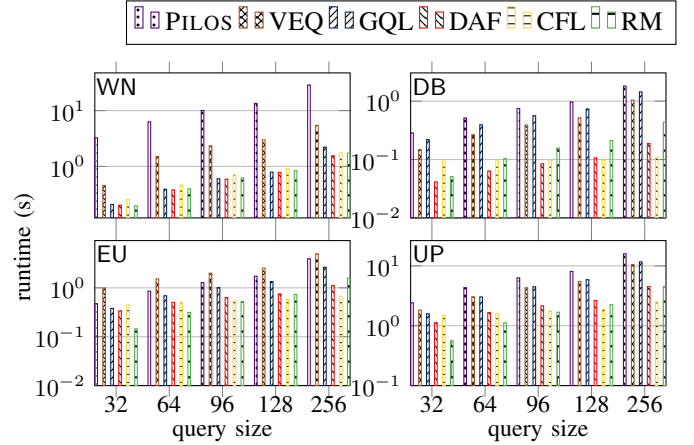


Figure 9: Average pruning time.

query size 256, VEQ has 5642, DAF 9 840, GQL 10 360, CFL 13 537 and RM has 23000 candidates on average, whereas PILOS has only 3820. At the same time, PILOS takes the most time at 2.5sec; VEQ takes 1.08s, GQL 1.5sec, DAF, RM and CFL less than 0.5sec each; nevertheless, these times are only about the filtering stage; as we will see, PILOS also has an advantage in terms of overall time for query answering.

The EU dataset yields an intermediate candidate space size, with around 20K remaining candidates for smaller queries and 30K for larger ones. PILOS has a significant pruning power advantage, around 7% over VEQ for smaller queries and up to 12% for larger ones; the gap is even higher for GQL, RM, DAF and CFL, 20–33% on query size 32 and 29% for

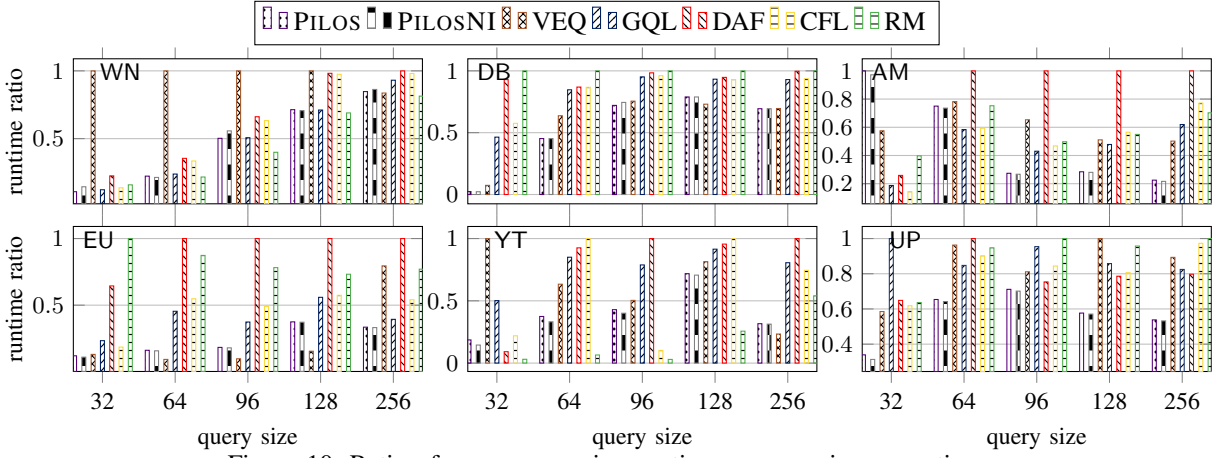


Figure 10: Ratio of query answering runtime over maximum runtime.

GQL, 46% for DAF and CFL and 54% for RM on query size 256. This search space differential is significant, as on query size 256 PILOS has to handle around 24 260 candidates, whereas CFL and DAF need to consider around 45 000. In terms of pruning-stage runtime on the same size, PILOS and VEQ needs around 4sec, RM 1.5sec, DAF 1sec, CFL 0.7s, and GQL 15sec; the difference is returned in terms of overall query answering time, as we will see.

On the largest dataset, UP, PILOS exhibits a significant advantage over other algorithms. The second best performer, VEQ displays 20% less pruning power on query size 32 and 41% on large queries. On small queries, GQL prunes 61% less, DAF 37%, CFL 58%, and RM 49%. The gap increases on larger queries, with GQL being 69% less effective, DAF 58%, CFL 69%, and RM 80%. In terms of pruning time, PILOS takes on average 18sec, VEQ and GQL 11 sec, DAF and RM 4.5sec, CFL takes 2.5sec on the largest queries.

Overall, these results show that the pruning rules and eigenvalue filtering of PILOS reduce candidates by up to 80%, while they require higher filtering time, which is compensated, as we will see, in terms of overall query answering efficiency.

D. Query Evaluation

To assess the end-to-end efficiency of PILOS, we report the number of queries that remain unresolved after an elapsed time limit and query answering time taking all queries into consideration.

Unresolved queries. Table IV shows the number of queries that remain unresolved after 10 minutes. PILOS stands out with regard to the amount of queries it resolves. PILOS has 13.40% unresolved queries where the second best algorithm, VEQ, 17.32% and GQL follows with 18.40%. Evidently, algorithms with higher pruning power resolve more queries. PILOS translates its pruning advantage into resolving more queries. On YT, which is relatively less challenging due to its sparsity, VEQ resolves 3 more queries than PILOS. Unresolved queries proliferate as query size grows, while only a few queries remain unresolved on query size 32 and 64. PILOS stands out as the only algorithm that resolves all queries of size 32. The effect of using an index is visible on WN,

where index-free PILOS (PILOSNI) resolves 12 fewer queries.

Table V reports the number of unresolved queries for the two best-performing algorithms, PILOS and VEQ, under a time constraint of 1 hour. PILOS consistently outperforms VEQ across datasets, resolving 3.97% more queries. This result verifies that allowing a larger timeout value benefits algorithms that invest more time in filtering, as examining fewer candidates leads to resolving more queries.

Table IV: Queries unresolved after 10 minutes.

Algo	Dataset						Sum	%
	WN	DB	AM	EU	YT	UP		
PILOS	445	803	48	110	22	180	1 608	13.40
PILOSNI	457	804	48	110	22	180	1 621	13.50
VEQ	655	826	149	143	19	287	2 079	17.32
GQL	496	1 073	168	163	50	259	2 209	18.40
DAF	588	1 127	313	381	68	263	2 740	22.83
CFL	578	1 087	210	211	51	292	2 429	20.24
RM	439	1 173	193	303	33	316	2 457	20.47

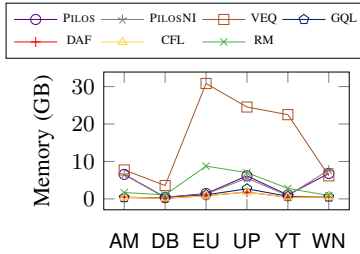
Table V: Queries unresolved after 1 hour.

Algo	Dataset						Sum	%
	WN	DB	AM	EU	YT	UP		
PILOS	410	777	37	98	15	168	1 505	12.54
VEQ	635	802	119	140	17	269	1 982	16.51

Query time. Next, we examine the total runtime to answer a query, including filtering and verification. **Figure 10** shows our results. We assign the time-limit value of 600 seconds (10 minutes) to unresolved queries. A higher time limit effectively raises the importance of the sheer amount of resolved queries, while a lower time limit confers more importance to the exact query time for queries resolved within the limit. In our setting, we have 400 queries per query size and a time limit of 10 minutes, thus each unresolved query adds $600/400 = 1.5$ seconds to the average runtime; however, if we allowed an 1-hour time limit, then each unresolved query would add 9 seconds to the average runtime, hence fewer unresolved queries would be needed to dominate the runtime. **Figure 10** presents our results on average query answering time. Due to the diversity of times across query sizes, we normalize each runtime by the maximum runtime value; thus, the slowest algorithm gets

a runtime ratio of 1. Notably, PILOS achieves the lowest average query answering times across data sets and query sizes, even as the query size grows. This result corroborates that the extra time PILOS spends for filtering is worthwhile, as it eventually leads to better query response times and to resolving *more* queries per se. PILOS’s strong pruning leads to stable performance across datasets and query sizes where other algorithms fail. For example, VEQ performs well in DB but terribly on WN, RM performs well on YT but terribly on DB. On WN, PILOS, GQL, and RM are the fastest for small queries and VEQ is the slowest. For large queries, PILOS, GQL, and VEQ are the fastest and DAF is the slowest. On DB, PILOS is the fastest for small queries, while for medium to large queries PILOS and VEQ are the fastest; RM is the slowest. On AM, PILOS is the slowest for small queries due to its high filtering time, but becomes the fastest as query size grows. The second best algorithm is GQL for small queries and VEQ for large queries. On EU, VEQ outperforms PILOS for small queries and PILOS outperforms VEQ for large queries with DAF being the slowest. On YT, the least challenging data set in terms of resolving queries, RM is the fastest algorithm for small queries and VEQ is the slowest. On UP, PILOS is the fastest, whilst GQL is the slowest for size 32, DAF for size 64, RM for size 96 and 256 and VEQ for size 128.

E. Memory Consumption



The figure on the left juxtaposes PILOS’s memory use to that of competitors. The simplest algorithms, i.e., DAF, CFL and GQL, have the lowest memory needs and VEQ the highest due to its complex enumeration phase, which reaches around 30GB on EU. RM requires around 9GB with EU, yet lower memory with other data. PILOS needs memory similar to VEQ with the AM and WN data, which yield many candidates, and lower needs with other data. PILOSNI requires slightly lower memory as it does not use an index, with the exception of the WN data, where PILOS uses less memory thanks to the high pruning power of the index.

F. Hyperparameter tuning

We now examine the effect on pruning power and runtime of hyperparameter α , i.e., the maximum size of subgraphs we evaluate by the extended safety rules. We impose this limit as larger subgraphs incur more costly evaluation of extended neighborhoods and eigenvalues, while subgraphs larger than that of the respective query vertex are also less likely to be pruned. **Figure 11** shows the results with α varying from 25 to 125. Larger subgraphs lead to higher pruning power at the cost of additional runtime. On some data, such as WN and DB, α affects the results heavily. In WN, where candidates are in the order of a million, a larger α dominates runtime and exceeds the 10-minute limit for large queries. Contrariwise,

on DB, where there are fewer candidates, it is worth using higher α to gain pruning power, especially for larger query sizes, at a small runtime cost. Overall, a smaller α helps resolve a query quickly, a larger α helps resolve hard queries.

The plot on the left portrays the pruning gain over NLF on WN vs. query size for several counts of indexed eigenvalues S . Pruning power rises with eigenvalues, yet most of the gain comes from the first few eigenvalues. Under limited memory, we can gain high pruning power even with a few stored eigenvalues per node. The plot also shows the power of pruning by the two-hop subgraph size $|N_k|$, lower than spectral pruning.

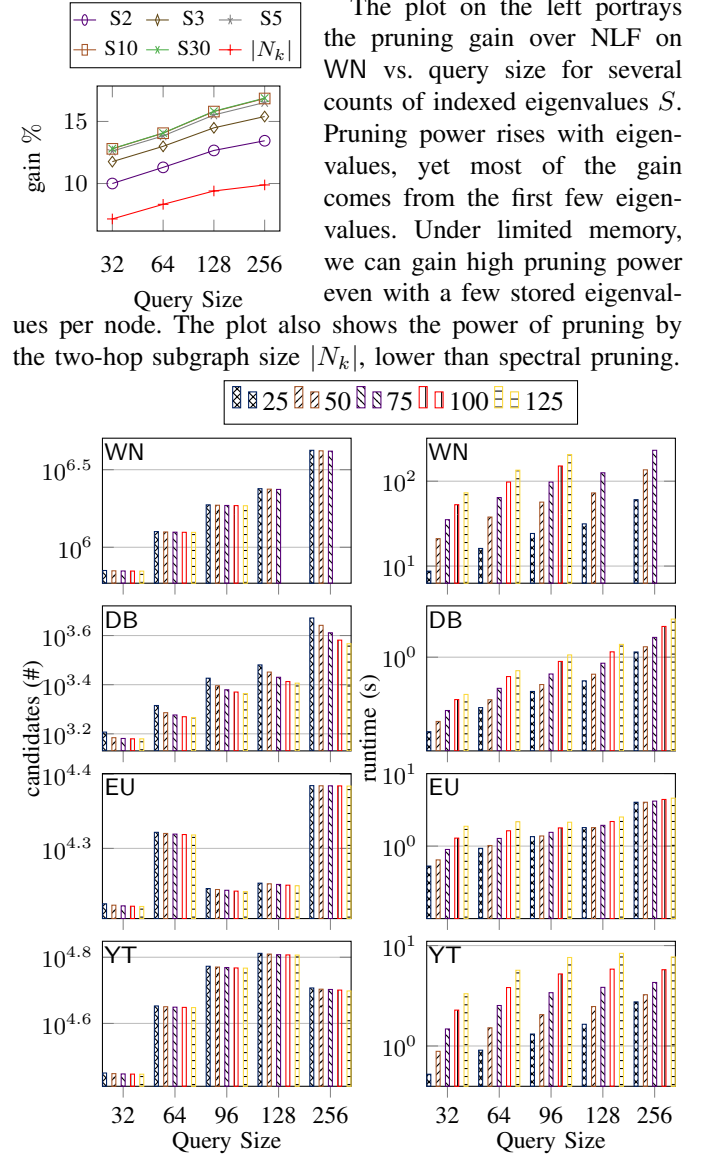


Figure 11: Tuning α ; left: candidates (#) after pruning; right: filtering runtime; results for queries finishing in 600sec.

VI. CONCLUSION

We proposed PILOS, a subgraph matching algorithm that leverages spectral graph information along with neighborhood rules to filter query matches, initially using a pre-built off-line index, and, *unprecedentedly* with respect to prior work, also with an online auxiliary data structure that represents candidates. Our exhaustive experimental study shows that, combined with state-of-the-art verification techniques, PILOS outperforms the state of the art in subgraph matching in pruning power, overall runtime, and number of queries resolved.

ACKNOWLEDGMENT

Work funded by Independent Research Fund Denmark (DFF).

REFERENCES

- [1] H. A. Rajapakse, "Small molecule inhibitors of the xiap protein-protein interaction," *Current topics in medicinal chemistry*, vol. 7, no. 10, pp. 966–971, 2007.
- [2] J. W. Raymond, E. J. Gardiner, and P. Willett, "Heuristics for similarity searching of chemical graphs using a maximum common edge subgraph algorithm," *J. of Chem. Inf. Comp.*, vol. 42, no. 2, pp. 305–316, 2002.
- [3] V. Bonnici, R. Giugno, A. Pulvirenti, D. Shasha, and A. Ferro, "A subgraph isomorphism algorithm and its application to biochemical data," *BMC bioinformatics*, vol. 14, no. 7, pp. 1–13, 2013.
- [4] B. Dost, T. Shlomi, N. Gupta, E. Ruppin, V. Bafna, and R. Sharan, "Qnet: a tool for querying protein interaction networks," *Journal of Computational Biology*, vol. 15, no. 7, pp. 913–925, 2008.
- [5] N. T. L. Tran, S. Mohan, Z. Xu, and C.-H. Huang, "Current innovations and future challenges of network motif detection," *Briefings in bioinformatics*, vol. 16, no. 3, pp. 497–525, 2015.
- [6] S. Zhang, S. Li, and J. Yang, "Gaddi: Distance index based subgraph matching in biological networks," in *EDBT*, ser. EDBT '09, 2009, p. 192–203.
- [7] N. Alon, P. Dao, I. Hajirasouliha, F. Hormozdiari, and S. C. Sahinalp, "Biomolecular network motif counting and discovery by color coding," *Bioinformatics*, vol. 24, no. 13, pp. i241–i249, 2008.
- [8] J. A. Grochow and M. Kellis, "Network motif discovery using subgraph enumeration and symmetry-breaking," in *RECOMB*. Springer, 2007, pp. 92–106.
- [9] T. A. Snijders, P. E. Pattison, G. L. Robins, and M. S. Handcock, "New specifications for exponential random graph models," *Sociological methodology*, vol. 36, no. 1, pp. 99–153, 2006.
- [10] W. Fan, "Graph pattern matching revised for social network analysis," in *ICDT*, 2012, pp. 8–21.
- [11] S. R. Kairam, D. J. Wang, and J. Leskovec, "The life and death of online groups: Predicting group growth and longevity," in *WSDM*, 2012, pp. 673–682.
- [12] T. Ma, S. Yu, J. Cao, Y. Tian, A. Al-Dhelaan, and M. Al-Rodhaan, "A comparative study of subgraph matching isomorphic methods in social networks," *IEEE Access*, vol. 6, pp. 66 621–66 631, 2018.
- [13] C. Solnon, G. Damiand, C. De La Higuera, and J.-C. Janodet, "On the complexity of submap isomorphism and maximum common submap problems," *Pattern Recognition*, vol. 48, no. 2, pp. 302–316, 2015.
- [14] G. Damiand, C. Solnon, C. De la Higuera, J.-C. Janodet, and É. Samuel, "Polynomial algorithms for subisomorphism of nd open combinatorial maps," *Computer Vision and Image Understanding*, vol. 115, no. 7, pp. 996–1010, 2011.
- [15] P. Foggia, G. Percannella, and M. Vento, "Graph matching and learning in pattern recognition in the last 10 years," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 28, no. 01, p. 1450001, 2014.
- [16] C. Vömel, F. De Lorenzi, S. Beer, and E. Fuchs, "The secret life of keys: on the calculation of mechanical lock systems," *SIAM Review*, vol. 59, no. 2, pp. 393–422, 2017.
- [17] A. Murray and B. Franke, "Compiling for automatically generated instruction set extensions," in *CGO*, 2012, pp. 13–22.
- [18] D. Bruschi, L. Martignoni, and M. Monga, "Detecting self-mutating malware using control-flow graph matching," in *DIMVA*. Springer, 2006, pp. 129–143.
- [19] M. Sevegnani and M. Calder, "Bigraphs with sharing," *Theoretical Computer Science*, vol. 577, pp. 43–73, 2015.
- [20] B. Chandramouli, J. Goldstein, and D. Maier, "High-performance dynamic pattern matching over disordered streams," *PVLDB*, vol. 3, no. 1-2, pp. 220–231, 2010.
- [21] Y. Yuan, D. Ma, Z. Wen, Z. Zhang, and G. Wang, "Subgraph matching over graph federation," *PVLDB*, vol. 15, no. 3, pp. 437–450, 2021.
- [22] S. Sun and Q. Luo, "Subgraph matching with effective matching order and indexing," *TKDE*, vol. 34, no. 1, pp. 491–505, 2020.
- [23] H. Jiang, H. Wang, S. Y. Philip, and S. Zhou, "Gstring: A novel approach for efficient search in graph databases," in *ICDE*, 2006, pp. 566–575.
- [24] C. R. Rivero and H. M. Jamil, "Efficient and scalable labeled subgraph matching using sgmatch," *KAIS*, vol. 51, no. 1, pp. 61–87, 2017.
- [25] X. Yan, P. S. Yu, and J. Han, "Graph indexing: a frequent structure-based approach," in *SIGMOD*, 2004, pp. 335–346.
- [26] S. Zhang, M. Hu, and J. Yang, "Treepi: A novel graph indexing method," in *ICDE*, 2007, pp. 966–975.
- [27] P. Zhao, J. X. Yu, and P. S. Yu, "Graph indexing: tree + delta $\geq$ graph," in *PVLDB*, 2007, pp. 938–949.
- [28] Z. Lin and Y. Bei, "Graph indexing for large networks: A neighborhood tree-based approach," *Knowledge-Based Systems*, vol. 72, pp. 48–59, 2014.
- [29] H. Shang, Y. Zhang, X. Lin, and J. X. Yu, "Taming verification hardness: an efficient algorithm for testing subgraph isomorphism," *PVLDB*, vol. 1, no. 1, pp. 364–375, 2008.
- [30] P. Zhao and J. Han, "On graph query optimization in large networks," *PVLDB*, vol. 3, no. 1-2, pp. 340–351, 2010.
- [31] K. Gouda and M. Hassaan, "Compressed feature-based filtering and verification approach for subgraph search," in *EDBT*, 2013, pp. 287–298.
- [32] W.-S. Han, J. Lee, M.-D. Pham, and J. X. Yu, "igraph: a framework for comparisons of disk-based graph indexing techniques," *PVLDB*, vol. 3, no. 1-2, pp. 449–459, 2010.
- [33] F. Katsarou, N. Ntarmos, and P. Triantafillou, "Performance and scalability of indexed subgraph query processing methods," *PVLDB*, vol. 8, no. 12, pp. 1566–1577, 2015.
- [34] J. R. Ullmann, "An algorithm for subgraph isomorphism," *J. ACM*, no. 1, p. 31–42, jan 1976.
- [35] V. Carletti, P. Foggia, A. Saggese, and M. Vento, "Challenging the time complexity of exact subgraph isomorphism for huge and dense graphs with vf3," *TPAMI*, vol. 40, no. 4, pp. 804–818, 2017.
- [36] F. Katsarou, N. Ntarmos, and P. Triantafillou, "Hybrid algorithms for subgraph pattern queries in graph databases," in *BigData*, 2017, pp. 656–665.
- [37] H. Kim, Y. Choi, K. Park, X. Lin, S.-H. Hong, and W.-S. Han, "Versatile equivalences: Speeding up subgraph query processing and subgraph matching," in *SIGMOD*, 2021, pp. 925–937.
- [38] B. Bhattarai, H. Liu, and H. H. Huang, "Ceci: Compact embedding cluster index for scalable subgraph matching," in *SIGMOD*, 2019, pp. 1447–1462.
- [39] F. Bi, L. Chang, X. Lin, L. Qin, and W. Zhang, "Efficient subgraph matching by postponing cartesian products," in *SIGMOD*, 2016, pp. 1199–1214.
- [40] S. Sun, X. Sun, Y. Che, Q. Luo, and B. He, "Rapidmatch: a holistic approach to subgraph query processing," *PVLDB*, vol. 14, no. 2, pp. 176–188, 2020.
- [41] A. Mhedhbi and S. Salihoglu, "Optimizing subgraph queries by combining binary and worst-case optimal joins," *Proc. VLDB Endow.*, vol. 12, no. 11, p. 1692–1704, jul 2019. [Online]. Available: <https://doi.org/10.14778/3342263.3342643>
- [42] S. Sun and Q. Luo, "Scaling up subgraph query processing with efficient subgraph matching," in *ICDE*, 2019, pp. 220–231.
- [43] —, "In-memory subgraph matching: An in-depth study," in *SIGMOD*, 2020, pp. 1083–1098.
- [44] Z. Zhang, Y. Lu, W. Zheng, and X. Lin, "A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction," *Proc. ACM Manag. Data*, vol. 2, no. 1, Mar. 2024. [Online]. Available: <https://doi.org/10.1145/3639315>
- [45] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, "Netlsd: hearing the shape of a graph," in *SIGKDD*, 2018, pp. 2347–2356.
- [46] J. Hermanns, K. Skitsas, A. Tsitsulin, M. Munkhoeva, A. Kyster, S. Nielsen, A. M. Bronstein, D. Mottin, and P. Karras, "Grasp: Scalable graph alignment by spectral corresponding functions," *TKDD*, vol. 17, no. 4, pp. 1–26, 2023.
- [47] L. Zou, L. Chen, J. X. Yu, and Y. Lu, "A novel spectral coding in a large graph database," in *EDBT*, 2008, pp. 181–192.
- [48] M. Han, H. Kim, G. Gu, K. Park, and W.-S. Han, "Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together," in *SIGMOD*, 2019, pp. 1429–1446.
- [49] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, "A (sub) graph isomorphism algorithm for matching large graphs," *TPAMI*, vol. 26, no. 10, pp. 1367–1372, 2004.
- [50] L. Zhu, Y. Yao, Y. Wang, X. Hei, Q. Zhao, W. Ji, and Q. Yao, "A novel subgraph querying method based on paths and spectra," *Neural Computing and Applications*, vol. 31, no. 9, pp. 5671–5678, 2019.
- [51] H. He and A. K. Singh, "Graphs-at-a-time: query language and access methods for graph databases," in *SIGMOD*, 2008, pp. 405–418.
- [52] W. Fan, J. Li, S. Ma, H. Wang, and Y. Wu, "Graph homomorphism revisited for graph matching," *PVLDB*, vol. 3, no. 1-2, pp. 1161–1172, 2010.
- [53] J. Cheng, J. X. Yu, and S. Y. Philip, "Graph pattern matching: A join/semijoin approach," *TKDE*, vol. 23, no. 7, pp. 1006–1021, 2010.

- [54] X. Wu, D. Theodoratos, N. Mamoulis, and M. Lan, "Evaluating hybrid graph pattern queries using runtime index graphs," *EDBT*, 2021.
- [55] W. Lin, X. Xiao, J. Cheng, and S. S. Bhowmick, "Efficient algorithms for generalized subgraph query processing," in *CIKM*, 2012, pp. 325–334.
- [56] S. Ma, Y. Cao, W. Fan, J. Huai, and T. Wo, "Strong simulation: Capturing topology in graph pattern matching," *TODS*, vol. 39, no. 1, pp. 1–46, 2014.
- [57] A. Khan, Y. Wu, C. C. Aggarwal, and X. Yan, "Nema: Fast graph search with label similarity," *PVLDB*, vol. 6, no. 3, pp. 181–192, 2013.
- [58] H.-R. Tan, C. Wang, S.-T. Wu, T.-Q. Wang, X.-Y. Zhang, and C.-L. Liu, "Proxy graph matching with proximal matching networks," in *AAAI*, vol. 35, no. 11, 2021, pp. 9808–9815.
- [59] I. Roy, V. S. B. R. Velugoti, S. Chakrabarti, and A. De, "Interpretable neural subgraph matching for graph retrieval," in *AAAI*, vol. 36, no. 7, 2022, pp. 8115–8123.
- [60] Z. Ying, A. Wang, J. You, C. Wen, A. Canedo, and J. Leskovec, "Neural subgraph matching," 2020.
- [61] J. Lee, W.-S. Han, R. Kasperovics, and J.-H. Lee, "An in-depth comparison of subgraph isomorphism algorithms in graph databases," *PVLDB*, vol. 6, no. 2, pp. 133–144, 2012.
- [62] D. Shasha, J. T. Wang, and R. Giugno, "Algorithmics and applications of tree and graph searching," in *PODS*, 2002, pp. 39–52.
- [63] M. F. Demirci, R. H. van Leuken, and R. C. Veltkamp, "Indexing through laplacian spectra," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 312–325, 2008.
- [64] J. Yan, X.-C. Yin, W. Lin, C. Deng, H. Zha, and X. Yang, "A short survey of recent advances in graph matching," in *ICMR*, 2016, pp. 167–174.
- [65] R. Singh, J. Xu, and B. Berger, "Pairwise global alignment of protein interaction networks by matching neighborhood topology," in *RECOMB*, 2007, pp. 16–31.
- [66] G. Zhu, X. Lin, K. Zhu, W. Zhang, and J. X. Yu, "Treespan: Efficiently computing similarity all-matching," in *SIGMOD*, 2012, p. 529–540.
- [67] D. B. West *et al.*, *Introduction to graph theory*. Prentice hall Upper Saddle River, 2001, vol. 2.
- [68] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," <http://eigen.tuxfamily.org>, 2010.
- [69] G. A. Miller, "Wordnet: A lexical database for english," *Commun. ACM*, vol. 38, no. 11, p. 39–41, nov 1995.
- [70] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," in *KDD Workshop on Mining Data Semantics*, 2012, pp. 1–8.
- [71] J. Leskovec, L. A. Adamic, and B. A. Huberman, "The dynamics of viral marketing," *ACM Trans. Web*, vol. 1, no. 1, p. 5–es, may 2007. [Online]. Available: <https://doi.org/10.1145/1232722.1232727>
- [72] P. Boldi and S. Vigna, "The WebGraph framework I: Compression techniques," in *TheWebConf*, Manhattan, USA, 2004, pp. 595–601.
- [73] P. Boldi, M. Rosa, M. Santini, and S. Vigna, "Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks," in *TheWebConf*, S. Srinivasan, K. Ramamritham, A. Kumar, M. P. Ravindra, E. Bertino, and R. Kumar, Eds., 2011, pp. 587–596.
- [74] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graphs over time: Densification laws, shrinking diameters and possible explanations," 2005, p. 177–187.
- [75] W.-S. Han, J. Lee, and J.-H. Lee, "Turboiso: towards ultrafast and robust subgraph isomorphism search in large graph databases," in *SIGMOD*, 2013, pp. 337–348.
- [76] F. Katsarou, N. Ntarmos, and P. Triantafillou, "Subgraph querying with parallel use of query rewritings and alternative algorithms," in *EDBT*, 2017, pp. 25–36.
- [77] X. Ren and J. Wang, "Exploiting vertex relationships in speeding up subgraph isomorphism over large graphs," *PVLDB*, vol. 8, no. 5, pp. 617–628, 2015.
- [78] Z. Sun, H. Wang, H. Wang, B. Shao, and J. Li, "Efficient subgraph matching on billion node graphs," *PVLDB*, vol. 5, no. 9, 2012.
- [79] L. Zhu and Q. Song, "A study of laplacian spectra of graph for subgraph queries," in *2011 IEEE 11th International Conference on Data Mining*. IEEE, 2011, pp. 1272–1277.